

Functions (Continued)

c

```
#include <stdio.h>

// Function declaration (prototype)
int add(int a, int b);
void greet(char name[]);
float calculateAverage(int values[], int size);

int main() {
    int result = add(5, 3);
    printf("5 + 3 = %d\n", result);

    greet("John");

    int scores[] = {85, 90, 78, 92, 88};
    float avg = calculateAverage(scores, 5);
    printf("Average score: %.2f\n", avg);

    return 0;
}

// Function definition
int add(int a, int b) {
    return a + b;
}

void greet(char name[]) {
    printf("Hello, %s!\n", name);
}

float calculateAverage(int values[], int size) {
    if (size <= 0) return 0.0f;

    int sum = 0;
    for (int i = 0; i < size; i++) {
        sum += values[i];
    }

    return (float)sum / size;
}
```

Parameter Passing

Pass by Value

In C, all parameters are passed by value, meaning a copy of the argument is passed to the function.

```
c
#include <stdio.h>

void modifyValue(int x) {
    x = x * 2; // Modifies the local copy, not the original
    printf("Inside function: x = %d\n", x);
}

int main() {
    int num = 10;

    printf("Before function call: num = %d\n", num);
    modifyValue(num);
    printf("After function call: num = %d\n", num); // num remains unchanged

    return 0;
}
```

Pass by Reference (using pointers)

To modify the original value, pass a pointer to the variable.

```
c
#include <stdio.h>

void modifyValue(int *x) {
    *x = *x * 2; // Modifies the original value
    printf("Inside function: *x = %d\n", *x);
}

int main() {
    int num = 10;

    printf("Before function call: num = %d\n", num);
    modifyValue(&num); // Pass the address of num
    printf("After function call: num = %d\n", num); // num is modified

    return 0;
}
```

Return Values

Functions can return a single value of any basic data type.

```
c

#include <stdio.h>

// Return an integer
int square(int x) {
    return x * x;
}

// Return a float
float celsiusToFahrenheit(float celsius) {
    return (celsius * 9.0f / 5.0f) + 32.0f;
}

// Return a character
char getGrade(int score) {
    if (score >= 90) return 'A';
    else if (score >= 80) return 'B';
    else if (score >= 70) return 'C';
    else if (score >= 60) return 'D';
    else return 'F';
}

int main() {
    int num = 5;
    float temp = 25.0f;
    int score = 85;

    printf("%d squared is %d\n", num, square(num));
    printf("%.1f°C is %.1f°F\n", temp, celsiusToFahrenheit(temp));
    printf("A score of %d gets a grade of %c\n", score, getGrade(score));

    return 0;
}
```

Arrays and Functions

Arrays have special behavior when passed to functions.

Passing Arrays to Functions

When an array is passed to a function, only the address of the first element is passed, not a copy of the entire array.

c

```
#include <stdio.h>

// Function that modifies an array
void modifyArray(int arr[], int size) {
    for (int i = 0; i < size; i++) {
        arr[i] *= 2; // Modifies the original array
    }
}

// Function that processes an array without modifying it
void printArray(const int arr[], int size) {
    printf("Array elements: ");
    for (int i = 0; i < size; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}

int main() {
    int numbers[5] = {10, 20, 30, 40, 50};

    printf("Before modification:\n");
    printArray(numbers, 5);

    modifyArray(numbers, 5);

    printf("After modification:\n");
    printArray(numbers, 5);

    return 0;
}
```

Passing Multidimensional Arrays

For 2D arrays, the second (and subsequent) dimensions must be specified in the function parameter.

C

```
#include <stdio.h>

// Function to print a 2D array
void print2DArray(int arr[][3], int rows) {
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < 3; j++) {
            printf("%d ", arr[i][j]);
        }
        printf("\n");
    }
}

// Function to find the sum of a 2D array
int sum2DArray(int arr[][3], int rows) {
    int total = 0;
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < 3; j++) {
            total += arr[i][j];
        }
    }
    return total;
}

int main() {
    int matrix[2][3] = {
        {1, 2, 3},
        {4, 5, 6}
    };

    printf("2D Array:\n");
    print2DArray(matrix, 2);

    int total = sum2DArray(matrix, 2);
    printf("Sum of all elements: %d\n", total);

    return 0;
}
```

Returning Arrays from Functions

C doesn't allow functions to directly return arrays, but there are several workarounds:

Return a Pointer to a Static Array

c

```
#include <stdio.h>

// Return a pointer to a static array (not recommended for general use)
int* generateNumbers() {
    static int numbers[5] = {10, 20, 30, 40, 50};
    return numbers;
}

int main() {
    int *result = generateNumbers();

    printf("Generated numbers: ");
    for (int i = 0; i < 5; i++) {
        printf("%d ", result[i]);
    }
    printf("\n");

    return 0;
}
```

Modify an Array Passed as Parameter

c

```
#include <stdio.h>

// Populate an array passed as parameter
void generateFibonacci(int arr[], int size) {
    if (size <= 0) return;

    arr[0] = 0;
    if (size == 1) return;

    arr[1] = 1;
    for (int i = 2; i < size; i++) {
        arr[i] = arr[i-1] + arr[i-2];
    }
}

int main() {
    int fibNumbers[10];

    generateFibonacci(fibNumbers, 10);

    printf("Fibonacci sequence: ");
    for (int i = 0; i < 10; i++) {
        printf("%d ", fibNumbers[i]);
    }
    printf("\n");

    return 0;
}
```

Use Dynamic Memory Allocation

c

```
#include <stdio.h>
#include <stdlib.h>

// Return a dynamically allocated array
int* createArray(int size, int value) {
    int *arr = (int*)malloc(size * sizeof(int));
    if (arr == NULL) {
        printf("Memory allocation failed!\n");
        exit(1);
    }

    for (int i = 0; i < size; i++) {
        arr[i] = value;
    }

    return arr;
}

int main() {
    int size = 5;
    int value = 7;

    int *result = createArray(size, value);

    printf("Created array: ");
    for (int i = 0; i < size; i++) {
        printf("%d ", result[i]);
    }
    printf("\n");

    // Don't forget to free the allocated memory
    free(result);

    return 0;
}
```

Array of Pointers

An array of pointers is an array where each element is a pointer.

Declaration and Initialization

c

```
#include <stdio.h>

int main() {
    // Array of integer pointers
    int a = 10, b = 20, c = 30;
    int *ptr_array[3];

    ptr_array[0] = &a;
    ptr_array[1] = &b;
    ptr_array[2] = &c;

    for (int i = 0; i < 3; i++) {
        printf("Value at ptr_array[%d]: %d\n", i, *ptr_array[i]);
    }

    // Array of string literals (char pointers)
    char *fruits[] = {
        "Apple",
        "Banana",
        "Cherry",
        "Orange"
    };

    printf("\nFruit list:\n");
    for (int i = 0; i < 4; i++) {
        printf("%d. %s\n", i+1, fruits[i]);
    }

    return 0;
}
```

Using Array of Pointers

Managing Variable-Length Strings

c

```
#include <stdio.h>
#include <string.h>

int main() {
    char *names[] = {
        "John",
        "Alice",
        "Bob",
        "Catherine"
    };

    // Sort names alphabetically (simple bubble sort)
    int n = 4;
    for (int i = 0; i < n-1; i++) {
        for (int j = 0; j < n-i-1; j++) {
            if (strcmp(names[j], names[j+1]) > 0) {
                // Swap pointers
                char *temp = names[j];
                names[j] = names[j+1];
                names[j+1] = temp;
            }
        }
    }

    printf("Sorted names:\n");
    for (int i = 0; i < n; i++) {
        printf("%s\n", names[i]);
    }

    return 0;
}
```

Array of Function Pointers

c

```
#include <stdio.h>

// Function declarations
int add(int a, int b) { return a + b; }
int subtract(int a, int b) { return a - b; }
int multiply(int a, int b) { return a * b; }
int divide(int a, int b) { return b != 0 ? a / b : 0; }

int main() {
    // Array of function pointers
    int (*operations[4])(int, int) = {add, subtract, multiply, divide};
    char *op_names[] = {"Addition", "Subtraction", "Multiplication", "Division"};

    int a = 10, b = 5;

    for (int i = 0; i < 4; i++) {
        printf("%s: %d\n", op_names[i], operations[i](a, b));
    }

    return 0;
}
```

Problem Solving using Arrays, Functions and Pointers

Let's explore some common problems that involve arrays, functions, and pointers.

Problem 1: Finding Maximum and Minimum in an Array

c

```
#include <stdio.h>

void findMinMax(int arr[], int size, int *min, int *max) {
    if (size <= 0) return;

    *min = *max = arr[0];

    for (int i = 1; i < size; i++) {
        if (arr[i] < *min) {
            *min = arr[i];
        }
        if (arr[i] > *max) {
            *max = arr[i];
        }
    }
}

int main() {
    int numbers[] = {34, 12, 89, 45, 23, 67, 5, 78};
    int size = sizeof(numbers) / sizeof(numbers[0]);
    int min, max;

    findMinMax(numbers, size, &min, &max);

    printf("Array: ");
    for (int i = 0; i < size; i++) {
        printf("%d ", numbers[i]);
    }
    printf("\n");

    printf("Minimum value: %d\n", min);
    printf("Maximum value: %d\n", max);

    return 0;
}
```

Problem 2: Matrix Transpose


```

#include <stdio.h>

// Function to transpose a matrix
void transposeMatrix(int original[][3], int transposed[][2], int rows, int cols) {
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            transposed[j][i] = original[i][j];
        }
    }
}

// Function to print a matrix
void printMatrix(int matrix[][3], int rows, int cols) {
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            printf("%d ", matrix[i][j]);
        }
        printf("\n");
    }
}

// Overloaded function to print the transposed matrix
void printTransposed(int matrix[][2], int rows, int cols) {
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            printf("%d ", matrix[i][j]);
        }
        printf("\n");
    }
}

int main() {
    int original[2][3] = {
        {1, 2, 3},
        {4, 5, 6}
    };

    int transposed[3][2];

    printf("Original Matrix (2x3):\n");
    printMatrix(original, 2, 3);

    transposeMatrix(original, transposed, 2, 3);

    printf("\nTransposed Matrix (3x2):\n");
    printTransposed(transposed, 3, 2);
}

```

```
    return 0;  
}
```

Problem 3: String Manipulation using Pointers


```
#include <stdio.h>

// Function to find the Length of a string using pointers
int stringLength(const char *str) {
    const char *ptr = str;
    while (*ptr != '\0') {
        ptr++;
    }
    return ptr - str;
}

// Function to copy a string using pointers
void stringCopy(char *dest, const char *src) {
    while (*src != '\0') {
        *dest = *src;
        dest++;
        src++;
    }
    *dest = '\0'; // Add null terminator
}

// Function to concatenate two strings using pointers
void stringConcat(char *dest, const char *src) {
    // Move to the end of dest
    while (*dest != '\0') {
        dest++;
    }

    // Copy src to the end of dest
    while (*src != '\0') {
        *dest = *src;
        dest++;
        src++;
    }
    *dest = '\0'; // Add null terminator
}

int main() {
    char str1[50] = "Hello";
    char str2[] = "World";
    char result[50];

    printf("str1: %s\n", str1);
    printf("str2: %s\n", str2);

    printf("Length of str1: %d\n", stringLength(str1));
}
```

```
    strcpy(result, str1);  
    printf("After copying str1 to result: %s\n", result);  
  
    stringConcat(result, " ");  
    stringConcat(result, str2);  
    printf("After concatenating: %s\n", result);  
  
    return 0;  
}
```

Problem 4: Array of Pointers to Structs


```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct {
    int id;
    char name[50];
    float salary;
} Employee;

// Function to display employee details
void displayEmployee(const Employee *emp) {
    printf("ID: %d, Name: %s, Salary: $%.2f\n",
        emp->id, emp->name, emp->salary);
}

int main() {
    // Create an array of Employee structures
    Employee employees[3] = {
        {101, "John Smith", 45000.0f},
        {102, "Alice Johnson", 52000.0f},
        {103, "Bob Williams", 48000.0f}
    };

    // Create an array of pointers to employees
    Employee *empPtrs[3];
    for (int i = 0; i < 3; i++) {
        empPtrs[i] = &employees[i];
    }

    printf("Employee Details:\n");
    for (int i = 0; i < 3; i++) {
        displayEmployee(empPtrs[i]);
    }

    // Sort employees by salary (using the pointers)
    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 2 - i; j++) {
            if (empPtrs[j]->salary > empPtrs[j+1]->salary) {
                // Swap pointers
                Employee *temp = empPtrs[j];
                empPtrs[j] = empPtrs[j+1];
                empPtrs[j+1] = temp;
            }
        }
    }
}

```

```
printf("\nEmployees Sorted by Salary:\n");  
for (int i = 0; i < 3; i++) {  
    displayEmployee(empPtrs[i]);  
}  
  
return 0;  
}
```

Problem Solving using Array of Pointers and Functions

Problem 1: Menu-Driven Program using Function Pointers


```
#include <stdio.h>
#include <stdlib.h>

// Function to find the sum of array elements
void findSum(int arr[], int size) {
    int sum = 0;
    for (int i = 0; i < size; i++) {
        sum += arr[i];
    }
    printf("Sum of elements: %d\n", sum);
}

// Function to find the average of array elements
void findAverage(int arr[], int size) {
    if (size == 0) {
        printf("Cannot calculate average of empty array\n");
        return;
    }

    int sum = 0;
    for (int i = 0; i < size; i++) {
        sum += arr[i];
    }
    printf("Average of elements: %.2f\n", (float)sum / size);
}

// Function to find the maximum element
void findMax(int arr[], int size) {
    if (size == 0) {
        printf("Array is empty\n");
        return;
    }

    int max = arr[0];
    for (int i = 1; i < size; i++) {
        if (arr[i] > max) {
            max = arr[i];
        }
    }
    printf("Maximum element: %d\n", max);
}

// Function to find the minimum element
void findMin(int arr[], int size) {
    if (size == 0) {
        printf("Array is empty\n");
    }
}
```

```

        return;
    }

    int min = arr[0];
    for (int i = 1; i < size; i++) {
        if (arr[i] < min) {
            min = arr[i];
        }
    }
    printf("Minimum element: %d\n", min);
}

int main() {
    int numbers[] = {12, 45, 67, 23, 9, 56, 89, 32};
    int size = sizeof(numbers) / sizeof(numbers[0]);

    // Array of function pointers
    void (*operations[])(int[], int) = {findSum, findAverage, findMax, findMin};
    char *op_names[] = {"Sum", "Average", "Maximum", "Minimum"};

    // Display menu
    printf("Array: ");
    for (int i = 0; i < size; i++) {
        printf("%d ", numbers[i]);
    }
    printf("\n\n");

    printf("Choose an operation:\n");
    printf("1. Calculate Sum\n");
    printf("2. Calculate Average\n");
    printf("3. Find Maximum\n");
    printf("4. Find Minimum\n");
    printf("0. Exit\n");

    int choice;
    do {
        printf("\nEnter your choice (0-4): ");
        scanf("%d", &choice);

        if (choice >= 1 && choice <= 4) {
            operations[choice-1](numbers, size);
        } else if (choice != 0) {
            printf("Invalid choice! Please try again.\n");
        }
    } while (choice != 0);

    printf("Program terminated.\n");
}

```



```
    return 0;  
}
```

Problem 2: Text Processing with Function Pointers


```

#include <stdio.h>
#include <string.h>
#include <ctype.h>

// Function to convert text to uppercase
void toUpperCase(char *text) {
    while (*text) {
        *text = toupper(*text);
        text++;
    }
}

// Function to convert text to Lowercase
void toLowerCase(char *text) {
    while (*text) {
        *text = tolower(*text);
        text++;
    }
}

// Function to capitalize first Letter of each word
void capitalizeWords(char *text) {
    int capitalize = 1; // Flag to indicate if next character should be capitalized

    while (*text) {
        if (isspace(*text)) {
            capitalize = 1;
        } else if (capitalize) {
            *text = toupper(*text);
            capitalize = 0;
        } else {
            *text = tolower(*text);
        }
        text++;
    }
}

// Function to reverse the text
void reverseText(char *text) {
    char *start = text;
    char *end = text + strlen(text) - 1;

    while (start < end) {
        // Swap characters
        char temp = *start;
        *start = *end;
        *end = temp;
    }
}

```

```

        *end = temp;

        // Move pointers
        start++;
        end--;
    }
}

int main() {
    char text[100];
    char original[100];

    printf("Enter a text string: ");
    fgets(text, sizeof(text), stdin);

    // Remove newline character if present
    size_t len = strlen(text);
    if (len > 0 && text[len-1] == '\n') {
        text[len-1] = '\0';
    }

    // Save original text
    strcpy(original, text);

    // Array of function pointers
    void (*textOperations[])(char *) = {
        toUpperCase,
        toLowerCase,
        capitalizeWords,
        reverseText
    };

    char *op_names[] = {
        "Convert to UPPERCASE",
        "Convert to lowercase",
        "Capitalize First Letter Of Each Word",
        "Reverse the text"
    };

    // Display menu
    printf("\nText Operations Menu:\n");
    for (int i = 0; i < 4; i++) {
        printf("%d. %s\n", i+1, op_names[i]);
    }
    printf("0. Exit\n");

    int choice;

```

```

do {
    printf("\nEnter your choice (0-4): ");
    scanf("%d", &choice);

    if (choice >= 1 && choice <= 4) {
        // Reset text to original
        strcpy(text, original);

        // Apply the selected operation
        textOperations[choice-1](text);

        // Display the result
        printf("Result: %s\n", text);
    } else if (choice != 0) {
        printf("Invalid choice! Please try again.\n");
    }
} while (choice != 0);

printf("Program terminated.\n");

return 0;
}

```

Storage Classes

Storage classes in C determine the scope, lifetime, and initialization of variables.

auto Storage Class

The `auto` storage class is the default for local variables. Variables are automatically created at the beginning of a block and destroyed at the end.

C

```
#include <stdio.h>

void demoAutoStorage() {
    auto int x = 10; // 'auto' is optional, as it's the default
    printf("Inside function: x = %d\n", x);
}

int main() {
    int y = 20; // 'auto' is implied
    {
        auto int z = 30;
        printf("Inside block: y = %d, z = %d\n", y, z);
    }
    // z is no longer accessible here

    demoAutoStorage();
    // x from demoAutoStorage() is not accessible here

    return 0;
}
```

register Storage Class

The `register` storage class suggests the compiler to store the variable in a CPU register for faster access. However, modern compilers optimize well, so this is often unnecessary.

c

```
#include <stdio.h>
#include <time.h>

int main() {
    // Example with register variable for loop counter
    clock_t start, end;
    double cpu_time_used;

    start = clock();
    register int i; // Suggest compiler to use a CPU register
    int sum = 0;
    for (i = 0; i < 1000000; i++) {
        sum += i;
    }
    end = clock();
    cpu_time_used = ((double) (end - start)) / CLOCKS_PER_SEC;

    printf("Sum: %d\n", sum);
    printf("Time taken: %f seconds\n", cpu_time_used);

    return 0;
}
```

static Storage Class

The `static` storage class has two uses:

1. For local variables, it preserves the value between function calls.
2. For global variables and functions, it limits the scope to the file.

c

```
#include <stdio.h>

// Function using static variable
void countCalls() {
    static int count = 0; // Initialized only once
    count++;
    printf("This function has been called %d time(s)\n", count);
}

// Static global variable
static int fileScope = 100; // Only accessible within this file

int main() {
    printf("Initial value of fileScope: %d\n", fileScope);

    // Demonstrate static local variable
    for (int i = 0; i < 5; i++) {
        countCalls();
    }

    return 0;
}
```

extern Storage Class

The `extern` storage class declares a variable that is defined elsewhere (typically in another file).

File: main.c

c

```
#include <stdio.h>

// External variable declared but not defined here
extern int sharedCounter;

// External function declaration
extern void incrementCounter();

int main() {
    printf("Initial counter value: %d\n", sharedCounter);

    incrementCounter();
    incrementCounter();

    printf("Final counter value: %d\n", sharedCounter);

    return 0;
}
```

File: counter.c

c

```
// Variable definition
int sharedCounter = 0;

// Function definition
void incrementCounter() {
    sharedCounter++;
}
```

Callback

A callback is a function that is passed as an argument to another function and is invoked at a specific time or when a specific event occurs.

Basic Callback Example

c

```
#include <stdio.h>

// Callback function type
typedef void (*CallbackFunc)(int);

// Functions that will be used as callbacks
void printSquare(int num) {
    printf("Square of %d is %d\n", num, num * num);
}

void printCube(int num) {
    printf("Cube of %d is %d\n", num, num * num * num);
}

// Function that takes a callback
void processNumbers(int arr[], int size, CallbackFunc callback) {
    for (int i = 0; i < size; i++) {
        callback(arr[i]);
    }
}

int main() {
    int numbers[] = {1, 2, 3, 4, 5};
    int size = sizeof(numbers) / sizeof(numbers[0]);

    printf("Processing with square callback:\n");
    processNumbers(numbers, size, printSquare);

    printf("\nProcessing with cube callback:\n");
    processNumbers(numbers, size, printCube);

    return 0;
}
```

Advanced Callback Example with Context


```

#include <stdio.h>

// Structure to hold callback context
typedef struct {
    void (*callback)(void *data, int value);
    void *data;
} CallbackInfo;

// Callback functions
void multiplyBy(void *factor, int value) {
    int multiplier = *((int *)factor);
    printf("%d * %d = %d\n", value, multiplier, value * multiplier);
}

void powerOf(void *exponent, int value) {
    int exp = *((int *)exponent);
    int result = 1;
    for (int i = 0; i < exp; i++) {
        result *= value;
    }
    printf("%d^%d = %d\n", value, exp, result);
}

// Function that uses the callback info
void processWithContext(int arr[], int size, CallbackInfo cbInfo) {
    for (int i = 0; i < size; i++) {
        cbInfo.callback(cbInfo.data, arr[i]);
    }
}

int main() {
    int numbers[] = {1, 2, 3, 4, 5};
    int size = sizeof(numbers) / sizeof(numbers[0]);

    // Set up callback with context for multiplication
    int factor = 10;
    CallbackInfo multiplyInfo = {multiplyBy, &factor};

    printf("Multiplying by %d:\n", factor);
    processWithContext(numbers, size, multiplyInfo);

    // Set up callback with context for exponentiation
    int exponent = 3;
    CallbackInfo powerInfo = {powerOf, &exponent};

    printf("\nRaising to power %d:\n", exponent);
}

```

```
    processWithContext(numbers, size, powerInfo);

    return 0;
}
```

Recursion

Recursion is a technique where a function calls itself to solve a problem.

Basic Recursion Example: Factorial

```
c

#include <stdio.h>

// Recursive function to calculate factorial
unsigned long factorial(int n) {
    // Base case
    if (n == 0 || n == 1) {
        return 1;
    }

    // Recursive case
    return n * factorial(n - 1);
}

int main() {
    for (int i = 0; i <= 10; i++) {
        printf("Factorial of %d = %lu\n", i, factorial(i));
    }

    return 0;
}
```

Fibonacci Sequence using Recursion

```
c

#include <stdio.h>

// Recursive function to calculate Fibonacci number
int fibonacci(int n) {
    // Base cases
    if
```